

Report on Traveling Salesperson

Rakesh Itani
Master's in Computer Science
Idaho State University
Pocatello, ID, 83209 USA
rakeshitani@isu.edu

April 2026

Abstract

This project report summarizes the use of the branch and bound algorithm to solve the traveling salesperson problem.

1 Introduction

Branch and bound is a method for solving optimization problems by breaking them down into smaller sub-problems and using a bounding function to eliminate sub-problems that cannot contain the optimal solution. The algorithm depends on efficient estimation of the lower and upper bounds of regions/branches of the search space. If no bounds are available, the algorithm degenerates to an exhaustive search.

The algorithm explores branches of the tree, which represent subsets of the solution set. Before enumerating the candidate solutions of a branch, the branch is checked against upper and lower estimated bounds on the optimal solution, and is discarded if it cannot produce a better solution than the best one found so far by the algorithm.

2 Pseudocode for branch and bound algorithm

```
1  def branchAndBound(self, time_allowance=60.0):
2      results = {"pruned": 0, "total": 0, "max": 0,
3                "count": 0}
4      cities = self._scenario.getCities()
5      ncities = len(cities)
6      foundTour = False
7      # use priority queue to keep track of next
8      # subproblem to search
9      priority_queue = []
10     # dictionary mapping priority hashing to sub
11     # problem
12     priority_dict = {}
13     cost, matrix =
14         self.create_initial_matrix(cities)
15     # initial BSSF using greedy approach
16     greedy_bssf = self.greedy()
17     if greedy_bssf["cost"] != np.inf:
18         foundTour = True
19         bssf = BSSF(
20             greedy_bssf["cost"], [city._index for
21                                   city in greedy_bssf["soln"].route]
22         )
23     else:
24         bssf = BSSF(np.inf, None)
25         start_time = time.time()
26         timed_out = False
27         initial_node = 0
28         cur_path = [initial_node]
29         initial_problem = SubProblem(cost, matrix,
30                                     cur_path)
31         results["total"] += 1
32         # update the priority queue
33         self.add_to_queue(initial_problem,
34                           priority_queue, priority_dict)
35     while len(priority_queue) != 0:
36         # sort the priority queue
```

```

30     priority_queue.sort(reverse=True)
31     cur_problem_score = priority_queue.pop()
32     cur_problem =
        priority_dict[cur_problem_score]
33     for j in range(ncities):
34         # Check timeout
35         if time.time() - start_time >
            time_allowance:
36             timed_out = True
37             break
38         # expand the subproblem
39         problemChild =
            self.expand_subproblem(cur_problem,
                j, results)
40         if problemChild is None:
41             continue
42         results["total"] += 1
43         if len(problemChild.path) == ncities:
44             if (
45                 problemChild.matrix[problemChild.path[-1],
                    problemChild.path[0]]
46                 != np.inf
47             ):
48                 foundTour = True
49                 problemChild.cost +=
                    problemChild.matrix[
50                     problemChild.path[-1],
                        problemChild.path[0]
51                 ]
52                 # update the BSSF
53                 if problemChild.cost <
                    bssf.cost:
54                     results["count"] += 1
55                     bssf.cost =
                        problemChild.cost
56                     bssf.route =
                        problemChild.path
57                 continue

```

```

58         # prune the child
59         else:
60             results["pruned"] += 1
61         else:
62             results["pruned"] += 1
63         elif problemChild.cost >= bssf.cost:
64             results["pruned"] += 1
65         else:
66             self.add_to_queue(problemChild,
67                               priority_queue, priority_dict)
67             if len(priority_queue) >
68                 results["max"]:
69                 results["max"] =
70                     len(priority_queue)
69         # check timeout
70         if timed_out:
71             break
72         citiesPath = [cities[index] for index in
73                       bssf.route]
73         bandbSolution = TSPSolution(citiesPath)
74         end_time = time.time()
75         # update the result
76         results["cost"] = bandbSolution.cost if
77             foundTour else math.inf
77         results["time"] = end_time - start_time
78         results["soln"] = bandbSolution
79         return results

```

Time complexity and space complexity

1. to add cities in priority queue takes $O(1)$ time and $O(n)$ spaces
2. outer loop iterates over all the cities $O(n)$ times
3. to sort and update the cities in priority queue it takes $O(n \log n)$ times and $O(n)$ spaces
4. inner loop iterates over all the expanded state $O(n!)$ times
5. while expanding the subproblem, generating successor states takes $O(n!)$ times and $O(n!)$ spaces and to compute the reduced cost matrix it takes $O(n^2)$ times and $O(1)$ space
6. to update the results it takes $O(n)$ times and $O(n)$ spaces

$$\begin{aligned} \text{Time_complexity} &= O(1) + O(n) * (O(n \log n) + O(n!) * (O(n!) + O(n^2) + O(n))) \\ &\approx O(n!) \end{aligned}$$

$$\begin{aligned} \text{Space_complexity} &= O(n) + O(n) * (O(n) + O(n!) + O(n!) + O(1) + O(n)) \\ &\approx O(n!) \end{aligned}$$

(1)

2.1 Pseudocode for greedy algorithm

```
1  def greedy(self, time_allowance=60.0):
2      cities = self._scenario.getCities()
3      cost, matrix =
4          self.create_initial_matrix(cities)  #
5          extract initial cost
6      results = {
7          "cost": math.inf,
8          "time": 0.0,
9          "soln": None,
10         "count": 0,
11         "max": None,
12         "total": None,
13         "pruned": None,
14     }
15     start_time = time.time()
16     # Iterate over possible starting nodes
17     for initial_node in range(len(cities)):
18         greedy_matrix = matrix.copy()
19         greedy_cost = cost
20         path = [initial_node]
21
22         while len(path) != len(cities):
23             # Check timeout
24             if time.time() - start_time >=
25                 time_allowance:
26                 return results
27
28             min_val, col_index =
29                 self.find_next_city(greedy_matrix,
30                                     path)
31
32             # No valid path found
33             if min_val == np.inf:
34                 break
35
36             self.update_greedy_matrix(greedy_matrix,
37                                       col_index, path)
38             greedy_cost += min_val
```

```

31         # complete tour found
32     else:
33         greedy_cost +=
            greedy_matrix[path[-1],
            initial_node]
34         cities_path = [cities[i] for i in
            path]
35         solution = TSPSolution(cities_path)
36         if solution.cost < results["cost"]:
37             results["cost"] = solution.cost
38             results["time"] = time.time() -
                start_time
39             results["soln"] = solution
40             results["count"] = 1
41     return results

```

Time complexity and space complexity

1. outer loop iterates over each possible starting node which takes $O(n)$ times
2. inner loop iterates over all cities to find the next city with minimum cost edge, taking $O(n)$ times
3. to find next city with minimum cost edge takes $O(n)$ times
4. to update the greedy matrix takes $O(1)$ times
5. to store greedy matrix takes $O(n^2)$ space

$$\begin{aligned}
 \text{Time_complexity} &= O(n) * O(n) * (O(n) + O(1)) \approx O(n^3) \\
 \text{Space_complexity} &= O(n^2)
 \end{aligned}
 \tag{2}$$

2.2 Pseudocode for helper functions

```
1  # function to expand subproblem
2  def expand_subproblem(self, subProblem, col,
   results):
3      if subProblem.matrix[subProblem.path[-1],
   col] == np.inf:
4          return None
5
6      matrix_copy = subProblem.matrix.copy()
7      matrix_copy[subProblem.path[-1]] = np.inf
8      matrix_copy[:, col] = np.inf
9      matrix_copy[col, subProblem.path[-1]] = np.inf
10
11     reductionCost =
12         self.reduce_cost_matrix(matrix_copy)
13
14     new_cost = (
15         reductionCost
16         + subProblem.cost
17         + subProblem.matrix[subProblem.path[-1],
18             col]
19     )
20     new_path = subProblem.path.copy()
21     new_path.append(col)
22     return SubProblem(new_cost, matrix_copy,
23         new_path)
```

```

1     # function to compute reduced cost matrix
2     def reduce_cost_matrix(self, matrix):
3         lower_bound = 0
4         for i in range(len(matrix)):
5             row_min = min(matrix[i, :])
6             if row_min != np.inf:
7                 matrix[i] -= row_min
8                 lower_bound += row_min
9         for j in range(len(matrix)):
10            col_min = min(matrix[:, j])
11            if col_min != np.inf:
12                matrix[:, j] -= col_min
13                lower_bound += col_min
14        return lower_bound

1     # function to compute initial matrix
2     def create_initial_matrix(self, cities):
3         initMatrix = np.full((len(cities),
4                                len(cities)), np.inf)
5         reductionCost = 0
6         for i, city_i in enumerate(cities):
7             for j, city_j in enumerate(cities):
8                 if i != j:
9                     initMatrix[i, j] =
                        city_i.costTo(city_j)
10        reductionCost =
            self.reduce_cost_matrix(initMatrix)
11        return reductionCost, initMatrix

1     # function to add items in queue
2     def add_to_queue(self, subProblem,
3                     priority_queue, priority_dict):
4         hash_value = subProblem.cost /
5                     len(subProblem.path)
6         priority_queue.append(hash_value)
7         priority_dict[hash_value] = subProblem

```

Time complexity and space complexity

1. to expand the subproblem it takes $O(n^2)$ times and $O(n^2)$ space to copy matrix
2. to compute the reduced cost matrix it takes $O(n^2)$ times and $O(n^2)$ space
3. to compute the initial cost matrix it takes $O(n^2)$ times and $O(n^2)$ space
4. to add items in queue it takes $O(1)$ time and $O(1)$ space

3 Observation and Results

3.1 Observation Table

Observation Table							
Cities	Seed	Running Time	Cost of Best Tour Found	Max# of stored states at a given time	#of BSSF updates	Total # of states created	Total # of states pruned
15	20	0.43 seconds	10534	70	17	12607	10558
16	902	0.99 seconds	7954	69	5	28363	24332
20	82	11.69 seconds	10845	147	6	238617	208826
25	840	60.00 seconds	11472	464	13	964964	811633
30	79	60.00 seconds	15954	2887	3	757690	544628
23	140	60.00 seconds	13182	208	13	1075192	916257
35	933	60.00 seconds	15036	2835	7	602696	492918
13	808	0.44 seconds	9173	45	8	15607	11932
19	365	0.63 seconds	10766	129	9	14591	12669
48	251	60.00 seconds	19964	1442	7	371449	301418

Figure 1: Observation Table for different number of cities and seed

In the figure (in figure 1), the seed represents the randomness, cities represents the number of cities, running time indicates time taken by program to compute branch and bound algorithm, cost of best tour indicates the optimal cost, max of stored states represents the number of item in priority queue, total number of states created and total number of states pruned.

The observation table reflects the exponential nature of the TSP problem i.e. the problem size increases as the number of states increases factorially. The number of pruned states also increases as the problem size increases. The algorithm effectively eliminates suboptimal paths using the lower bound criteria. This helps to reduce the the search space making algorithm more efficient.

However, we can still see the increment in time taken to solve the problem with increased problem size because increased problem size also increases the number of states to explore. This indicates that, the time complexity grows rapidly with problem size due to the factorial growth in the number of states to explore.

Mechanism for state space search

A branch and bound algorithm was used to solve the TSP problem. Unlike the brute force method, this method was found to be effective. First, BSSF was computed using the greedy approach, then the state was branched into multiple smaller subproblems. For each of these subproblems, the lower bound was computed and then compared to the initial BSSF. All those subproblems with lower

bound greater than BSSF would be pruned. This allows the algorithm to work in two steps: Branching the problem and bounding the subproblems.

To get the solution more efficiently, using greedy approach to get the BSSF potentially reduces the search space. Similarly, the lower bound criterion helps to eliminate the suboptimal paths early which reduces the number of states to explore. Also the use of priority queue helps to identify the potential subproblem that might give optimal solution. The cities (subproblems) are sorted in the priority queue which helps the program to choose the best subproblem and eventually get the optimal solution.

3.2 Results and Analysis

1. Priority queue - For insertion and removal of item from priority queue, it takes $O(n \log n)$ time and these operations are performed for every expanded state, which happens at most $O(n!)$ times. Thus, total time complexity is $O(n!)$ and space complexity is $O(n!)$
2. Search States - For generating the successor states, we need to iterate through every cities, which is $O(n)$ times and the operation is performed for every state, which happens at most $O(n!)$ times. Thus, total time complexity is $O(n!)$
3. Reduced Cost Matrix - To update the reduced cost matrix, it takes $O(n^2)$ times and space complexity is $O(n^2)$
4. BSSF Initialization - We use greedy approach i.e. choose the tour based on the minimal edge cost from the starting city. So, the time and space complexity is $O(n^2)$
5. Expanding on SearchState into others - Expanding the subproblem involves generating successor states and updating the priority queue. So, the time complexity is $O(n!)$

The data structure used to represent the states are matrix.

The priority queue data structure I used is the dictionary. Using the subproblem, we compute the hash value and it is added to priority dictionary as key and value pair.

To initialize the BSSF, I used the greedy approach. The starting city is selected randomly then the next city is chosen based on the next closest edge of the neighboring cities.

4 Conclusion

A branch and bound algorithm was coded to solve the traveling salesperson problem. Initial BSSF was computed using the greedy approach which was followed by the checking of criteria i.e compares the lower bound against sub optimal paths and prunes the state.

The branch and bound algorithm created in this project works in two steps: expanding the subproblems then enforcing certain boundaries i.e. bound, the program reflects the exponential nature of the TSP problem. As the problem size increases, the number of possible permutations increases factorially. Then, the program prunes the states that maynot lead to the optimal solution.